# LLMSHIELD: A PROMPT INJECTION DETECTION AND FIREWALL SYSTEM FOR LLM-INTEGRATED APPLICATIONS

**A PROJECT REPORT**

***Submitted by***


**ADHIYAN PERIYAR R P 2116231901002**

**AKASH DURAI V 2116231901004**


***in partial fulfillment for the award of the degree of***

**BACHELOR OF ENGINEERING**

***in***

**COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)**








**RAJALAKSHMI ENGINEERING COLLEGE**

**ANNA UNIVERSITY: CHENNAI 600 025**


**NOVEMBER 2026**

# RAJALAKSHMI ENGINEERING COLLEGE
# CHENNAI

## BONAFIDE CERTIFICATE

Certified that this Project report titled **"LLMSHIELD: A PROMPT INJECTION DETECTION AND FIREWALL SYSTEM FOR LLM-INTEGRATED APPLICATIONS"** is the Bonafide work of **"ADHIYAN PERIYAR R P (2116231901002), AKASH DURAI V (2116231901004)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

06/10/26

**SIGNATURE**

Dr. P. Tamilselvi, M.E., Ph.D.,

**HEAD OF THE DEPARTMENT**

**PROFESSOR**

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

06/10/26

**SIGNATURE**

Mrs. Janani I, M. Tech

**SUPERVISOR**

**ASSISTANT PROFESSOR**

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 07/10/2026

**Internal Examiner**

7/10/26

**External Examiner**

# RAJALAKSHMI ENGINEERING COLLEGE

# CHENNAI

## BONAFIDE CERTIFICATE

Certified that this Project report titled **"LLMSHIELD: A PROMPT INJECTION DETECTION AND FIREWALL SYSTEM FOR LLM-INTEGRATED APPLICATIONS"** is the Bonafide work of **"ADHIYAN PERIYAR R P (2116231901002), AKASH DURAI V (2116231901004)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.


**SIGNATURE**

Dr. P. Tamilselvi, M.E., Ph.D.,

**HEAD OF THE DEPARTMENT**

**PROFESSOR**

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

**SIGNATURE**

Mrs. Janani I, M. Tech

**SUPERVISOR**

**ASSISTANT PROFESSOR**

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105


Submitted to Project Viva-Voce Examination held on _____

**Internal Examiner** **External Examiner**

# ABSTRACT

Large Language Models (LLMs) are increasingly used in applications such as chatbots, document summarizers, and autonomous assistants, but their integration introduces security risks, particularly prompt injection attacks, identified as LLM01 by OWASP. This project presents LLMShield, a middleware firewall that detects malicious prompts and indirect injections from external content using a hybrid approach combining rule-based pattern matching, including common obfuscation detection, and a TF-IDF-based Support Vector Machine (SVM) classifier. Their outputs are combined using weighted risk scoring to provide four decisions: ALLOW, FLAG, SANITIZE, and BLOCK, along with an explanation of the detection result. LLMShield was evaluated on a custom dataset of 143 samples, achieving 82% accuracy, precision, recall, and F1-score on the held-out test set, and achieved a 94.4% defense success rate on 18 independent red-team cases. The results demonstrate that a lightweight and locally trained hybrid approach can provide explainable protection against prompt injection attacks without requiring an additional LLM-based validator.

## ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mrs. Janani I, Assistant Professor (AP)(SS)** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

**ADHIYAN PERIYAR R - 2116231901002**

**AKASH DURAI V – 2116231901004**

## TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | iii |
| | **LIST OF TABLES** | vii |
| | **LIST OF FIGURES** | viii |
| **1** | **INTRODUCTION** | **1** |
| | 1.1 BACKGROUND | |
| | 1.2 PROBLEM STATEMENT | |
| | 1.3 OBJECTIVES | |
| | 1.4 SCOPE | |
| | 1.5 SECURITY REQUIREMENTS | |
| | 1.6 SDG ALIGNMENT | |
| **2** | **LITERATURE REVIEW** | **5** |
| | 2.1 EXISTING SYSTEM WITH SECURITY APPROACHES | |
| | 2.2 RESEARCH GAPS | |
| | 2.3 PROPOSED SYSTEM | |
| | 2.3.1 OVERVIEW OF THE PROPOSED SYSTEM | |
| | 2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM | |
| | 2.3.3 ADVANTAGES OF PROPOSED SYSTEM | |
| **3** | **SYSTEM DESIGN** | **9** |
| | 3.1 SYSTEM ARCHITECTURE | |
| | 3.2 MODULE DESIGN | |
| | 3.3 DATA FLOW | |
| | 3.4 THREAT MODEL AND ATTACK SURFACE | |

3.5 SECURITY CONTROLS

**4 SYSTEM DESCRIPTION 14**

4.1 DATA COLLECTION

4.2 DATA PREPROCESSING

4.2.1 ALGORITHMS

4.2.2 RESULTS AND DISCUSSION

**5 CONCLUSION** 19

21

**REFERENCES**

## LIST OF TABLES


| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| 1.1 | SDG Alignment | 4 |
| 3.1 | Module Design and Responsibilities | 10 |
| 4.1 | Dataset Category Distribution | 14 |
| 4.2 | Risk Score to Decision Mapping | 15 |
| 4.3 | SVM Classifier Evaluation (Held-out Test Split) | 16 |
| 4.4 | Red-Team Adversarial Benchmark Results | 17 |

## LIST OF FIGURES


| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| 3.1 | System Architecture Diagram | 9 |
| 3.2 | Use Case Diagram | 11 |
| 3.3 | Activity Diagram | 11 |
| 3.4 | Sequence Diagram | 12 |
| 4.1 | Confusion Matrix — SVM Classifier | 16 |
| 4.2 | Red-Team Adversarial Benchmark Results Chart | 17 |

# CHAPTER 1

# INTRODUCTION

## 1.1 BACKGROUND

Large Language Models (LLMs) such as GPT-4 and Claude have rapidly become foundational components of modern software systems, powering chatbots, document summarizers, coding assistants, and autonomous agents. This architectural flexibility, however, has introduced a new class of security vulnerability. Unlike traditional software systems where code and data are cleanly separated, LLMs process instructions and data through the same channel — natural language text.

This means an LLM currently has no reliable, built-in mechanism to distinguish between a legitimate instruction from an authorized developer or user and a malicious instruction embedded within content it is merely asked to process. This vulnerability is known as prompt injection, and it has been formally recognized as the single most critical security risk for LLM-integrated applications, ranked LLM01 in the OWASP Top 10 for Large Language Model Applications.

Prompt injection attacks fall into two broad categories: direct injection, where an attacker types malicious instructions straight into a chat interface, and indirect injection, where the malicious instruction is hidden inside external content — a resume being screened, a webpage being summarized, or an email being processed — that the LLM reads as part of its task. Indirect injection is particularly dangerous because the end user issuing the original request may have no visibility into the compromised content at all.

Unlike well-studied web vulnerabilities such as SQL Injection or Cross-Site Scripting (XSS), which have mature, widely adopted defense mechanisms, prompt injection defenses remain an emerging and unstandardized field. Existing mitigation strategies — including input filtering, delimiter-based separation, and instruction-based defenses — have each been individually shown to be insufficient against

sufficiently adaptive attacks. This project addresses this gap by designing and implementing LLMSHIELD, a middleware security firewall that inspects prompts and retrieved content before they reach an LLM, combining interpretable rule-based detection with a trainable machine learning classifier through a graded, explainable decision engine.

## 1.2 PROBLEM STATEMENT

AI systems that integrate Large Language Models can be manipulated through prompt injection attacks, wherein an attacker embeds hidden or disguised instructions within text supplied to the model — either directly through user input or indirectly through external content such as a webpage, document, or email that the AI is asked to process. A successful attack can cause the LLM to disregard its intended task, leak confidential system information, or perform actions that favor the attacker rather than the legitimate user or application owner. Currently, no lightweight, explainable, and easily deployable detection system exists that can be placed in front of any LLM-integrated application — regardless of which LLM it uses — to defend against both direct and indirect forms of this attack.

## 1.3 OBJECTIVES

- To design and implement a middleware firewall capable of detecting prompt injection attacks before they reach an LLM, covering both direct (user-typed) and indirect (document/webpage-embedded) attack vectors.
- To build a rule-based detection engine capable of identifying known attack patterns, including obfuscated variants such as Base64 encoding, leetspeak substitution, and character spacing tricks.
- To construct a custom-labeled dataset spanning benign queries and multiple distinct attack categories, and to train a Support Vector Machine (SVM) classifier on TF-IDF text features to detect paraphrased or previously unseen attacks.

- To combine the rule-based and machine-learning detectors using a weighted ensemble risk scoring algorithm that produces a graded decision — ALLOW, FLAG, SANITIZE, or BLOCK — rather than a simple binary response.
- To provide a human-readable explanation for every firewall decision, so the system's behavior is interpretable rather than a black box.
- To evaluate the system's robustness through an adversarial red-teaming benchmark comprising jailbreak and evasion techniques not present in the training dataset.
- To develop a working demonstration, including a REST API middleware and a browser-based dashboard, showing the system operating on both direct-input and indirect (document-based) attack scenarios.

## 1.4 SCOPE

The scope of this project is limited to text-based prompt injection detection for LLM-integrated applications. The system operates as an external middleware layer and does not require modifying the underlying LLM or its training process, making it deployable in front of any third-party or black-box LLM API. The current implementation focuses on English-language attack patterns and evaluates detection using a custom-built dataset and an independent adversarial red-teaming benchmark. The system is demonstrated using a simulated ("mock") LLM to illustrate before/after protection behavior; integration with a live, production LLM API is identified as future work. Multi-turn conversational context tracking across multiple separate messages is also outside the current scope, with each request evaluated independently.

## 1.5 SECURITY REQUIREMENTS

- Confidentiality: The system must prevent indirect injection attacks from causing the LLM to leak confidential information, such as system prompts, internal configuration, or sensitive data embedded in its context.

- Integrity: The system must ensure that the LLM's intended behavior cannot be overridden by instructions embedded in untrusted user input or retrieved content.
- Availability: Detection and scoring must be computationally lightweight (CPU-only, no external API dependency) so that the firewall does not introduce unacceptable latency or become a single point of failure for the protected application.
- Explainability: Every decision made by the system must be accompanied by a human-readable justification, since an opaque security control is difficult for developers or auditors to trust and verify.
- Defense-in-depth: No single detection technique is assumed to be sufficient; the system combines two independent detection signals (rule-based and machine-learning) so that a weakness in one is compensated by the other.
- Fail-safe defaults: Ambiguous or moderately suspicious input is not silently allowed; it is flagged or sanitized rather than passed through unexamined, in keeping with the principle of least privilege applied to LLM-generated actions.

## 1.6 SDG ALIGNMENT

| S. No. | SDG No. & Goal | Relevant SDG Target | Project Contribution / Alignment |
|---|---|---|---|
| 1 | SDG 9 — Industry, Innovation and Infrastructure | 9.5 — Enhance scientific research and technological capabilities | Contributes an applied security technique (hybrid rule-based + ML detection with explainable risk scoring) to the emerging field of AI/LLM security engineering. |
| 2 | SDG 16 — Peace, Justice and Strong Institutions | 16.10 — Ensure public access to information and protect fundamental freedoms | Prevents unauthorized data leakage and manipulation of AI systems, protecting the integrity and trustworthiness of information delivered through AI-integrated applications. |
| 3 | SDG 4 — Quality Education | 4.4 — Increase the number of people with relevant technology skills | Undertaken as an academic exercise building practical skills in cybersecurity, machine learning, and secure software engineering |

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

A review of current literature and industry practice reveals three broad categories of existing approaches to the prompt injection problem.

**(i) Attack Methodology Research**

Liu et al. [1] introduced HOUYI, a black-box prompt injection attack methodology composed of three components — a Framework Component that blends with the application's normal functionality, a Separator Component that forces a context break, and a Disruptor Component that carries the malicious payload. HOUYI was evaluated against 36 real-world commercial LLM-integrated applications and successfully compromised 31 of them (an 86.1% success rate), including well-known services such as Notion. Critically, the authors demonstrated that HOUYI could bypass six previously proposed defense strategies — Instruction Defense, Post-Prompting, Random Sequence Enclosure, Sandwich Defense, XML Tagging, and Separate LLM Evaluation — showing that static, single-layer defenses are insufficient against adaptive attackers.

**(ii) Detection-Based Defences**

Muliarevych [2] proposed a layered Filter architecture consisting of a Filtering Request Processor, a Prompt Analyzer, and an Attack Validator subsystem. The study compared a lightweight Fuzzy Search baseline against LLM-based validators built on GPT-3 and GPT-4, evaluated across five attack categories using 800 test prompts per run. The GPT-4-based Attack Validator achieved detection rates ranging from 76% to 97% depending on the attack category, substantially outperforming the Fuzzy Search baseline in accuracy and precision. However, the Fuzzy Search method was considerably faster (71ms average response time) compared to the GPT-3 and GPT-4 validators (289ms and 348ms average, respectively), highlighting a direct trade-off

between detection accuracy and computational cost/latency when an LLM itself is used as the detector.

**(iii) Prevention-Based Defenses**

Suo [3] proposed Signed-Prompt, a fundamentally different strategy that does not attempt to detect malicious instructions but instead prevents them from being executed. The method works by having an Encoder "sign" trusted instructions from authorized users — replacing them with character sequences that rarely occur in natural language — before they reach the LLM, which is separately adjusted (via prompt engineering or fine-tuning) to execute only signed instructions. Experiments across four test groups (Direct, Multilingual, Varied Expression, and Implication phrasing) showed a 0% attacker success rate, with up to 100% correctness maintained for legitimate signed instructions using the prompt-engineered variant.

**(iv) Industry Risk Classification**

The OWASP Foundation's Top 10 for Large Language Model Applications [4] formally ranks Prompt Injection as LLM01 — the single highest-priority security risk for LLM-integrated applications — establishing the real-world urgency of the problem this project addresses.

**(v) Hybrid Prompt Injection Detection**

Chen et al. [5] proposed a defense approach for prompt injection attacks that specifically considers malicious instructions embedded within external or retrieved data. The approach leverages attack techniques to improve the detection and defense process against indirect prompt injection attacks. The study evaluates the proposed method against existing training-free defenses and demonstrates improved protection against prompt injection attacks.

**(vi) Ensemble-Based Prompt Injection Detection**

Srinivasan et al. [6] investigated prompt injection detection using a hybrid approach that combines rule-based detection with machine-learning-based

classification. The study demonstrates that combining deterministic security rules with learned model predictions can provide a more comprehensive detection mechanism than relying on a single detection method. The outputs of the detection components are integrated through a decision-fusion approach to improve prompt injection detection.

## 2.2 RESEARCH GAPS

- Attack-only research provides no deployable defense. HOUYI and similar attack methodology papers systematically expose vulnerabilities but do not propose a corresponding detection or defense system.
- High-accuracy detection currently depends on costly LLM-based validation. The strongest published detection results (76%–97%) rely on using a second LLM as a validator for every incoming prompt, which is expensive, slower (289–348ms average latency), and creates dependency on a third-party LLM provider.
- Lightweight approaches sacrifice too much accuracy. The Fuzzy Search baseline, while fast, showed the lowest precision and recall among tested configurations.
- Prevention-based approaches require modifying the application or LLM itself. Signed-Prompt achieves excellent results but is not deployable as an external, plug-in firewall in front of a black-box, third-party LLM API.
- Limited attention to graded, explainable responses. Most surveyed systems reduce their decision to a binary classification with no graded response mechanism or human-readable explanation.

## 2.3 PROPOSED SYSTEM

### 2.3.1 Overview of the Proposed System

To address the gaps identified above, this project proposes LLMSHIELD, a middleware security firewall that combines two independent, complementary

detection signals — a rule-based pattern matching engine and a locally-trained machine learning classifier — through a weighted ensemble risk scoring algorithm. Unlike systems that depend on an LLM as a validator, LLMSHIELD's ML component is a lightweight Support Vector Machine trained on TF-IDF text features, which is CPU-trainable in seconds and requires no external API calls at inference time.

### 2.3.2 Objectives of the Proposed System

- To detect both direct and indirect prompt injection attacks using a hybrid detection approach.
- To avoid dependency on external LLM APIs for the detection mechanism itself, ensuring low latency and zero inference cost.
- To provide graded, explainable decisions rather than a binary classification.
- To validate detection robustness against novel, previously unseen adversarial techniques through dedicated red-team testing.

### 2.3.3 Advantages of Proposed System

- Lightweight and fast: CPU-only training and inference, with no GPU or external API dependency.
- Explainable: Every decision includes a stated reason, supporting auditability.
- Graded response: Distinguishes between content that should be blocked, sanitized, or merely flagged.
- Deployment-agnostic: Operates as an external middleware layer compatible with any LLM-integrated application.
- Empirically validated: Evaluated on held-out data and an independent adversarial red-teaming benchmark.

# CHAPTER 3

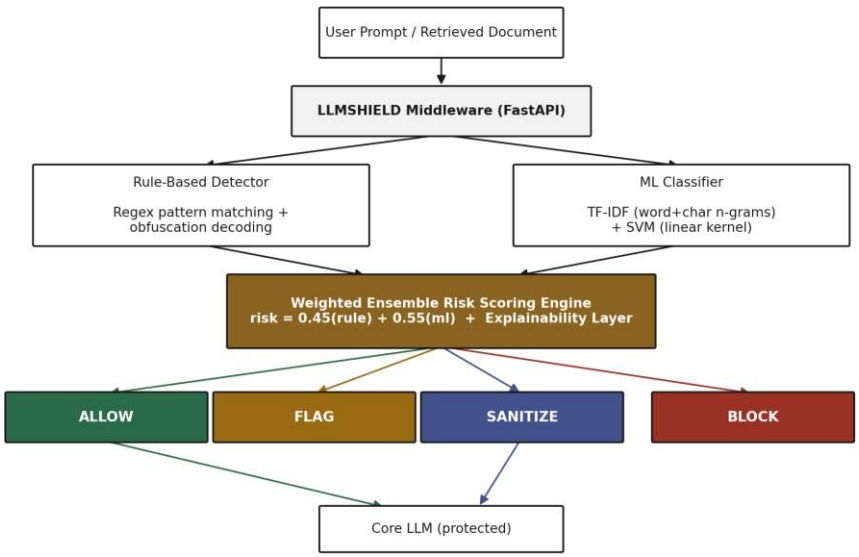
# SYSTEM DESIGN

## 3.1 SYSTEM ARCHITECTURE

LLMShield is designed as a middleware layer that intercepts all text — whether typed directly by a user or retrieved from an external source such as a document or webpage — before it is allowed to reach the core LLM. The architecture follows a pipeline of four stages: intake, parallel detection, ensemble scoring, and graded routing.

When a request arrives at the LLMSHIELD Middleware (implemented as a FastAPI service), it is simultaneously passed to two independent detectors: the Rule-Based Detector and the ML Classifier. Both detectors output a confidence score, which are combined by the Weighted Ensemble Risk Scoring Engine into a single risk score between 0 and 1. This score is mapped to one of four graded decisions — ALLOW, FLAG, SANITIZE, or BLOCK — each accompanied by a human-readable explanation. Only content that is ALLOWED or SANITIZED is forwarded onward to the protected core LLM; BLOCKED content never reaches it.

**Figure 3.1 — System Architecture Diagram**



## 3.2 MODULE DESIGN

**Table 3.1 — Module Design and Responsibilities**

| Module | Responsibility |
|---|---|
| Rule-Based Detector | Normalizes input text (decodes obfuscation) and matches it against a curated regex pattern library spanning five attack categories. Returns matched categories and a rule-based confidence score. |
| ML Classifier | Vectorizes input text using TF-IDF (word + character n-grams) and classifies it using a trained linear-kernel SVM. Returns a probability score for the "malicious" class. |
| Weighted Ensemble Engine | Combines the rule-based confidence and ML probability into a single risk score using a weighted formula, and maps the score to a graded decision. |
| Explainability Layer | Generates a human-readable narrative describing which detection signals contributed to a decision and why. |
| Mock LLM | Simulates a downstream LLM application, used to demonstrate before/after protection behaviour without requiring a live API key. |
| FastAPI Middleware | Exposes the firewall's functionality via REST endpoints (/check, /simulate, /health) so it can be integrated into any application. |
| Dashboard (Frontend) | Browser-based interface for live prompt inspection, indirect-injection simulation, and red-team benchmark visualization. |

## 3.3 DATA FLOW / UML DIAGRAMS

The data flow through LLMShield proceeds as follows:

(1) a user or an external application submits a prompt, either as direct input or as a source-tagged document/webpage

(2) the FastAPI middleware receives the request and forwards the raw text simultaneously to the Rule-Based Detector and the ML Classifier

(3) each detector independently returns a confidence score and, for the rule engine, a list of matched categories

(4) the Weighted Ensemble Engine combines these into a risk score and decision

(5) the Explainability Layer attaches a reasoning string to the decision

(6) the middleware returns the full result to the caller and, for ALLOW/SANITIZE decisions, the (possibly cleaned) text is forwarded to the downstream LLM.

**Figure 3.2 — Use Case Diagram**

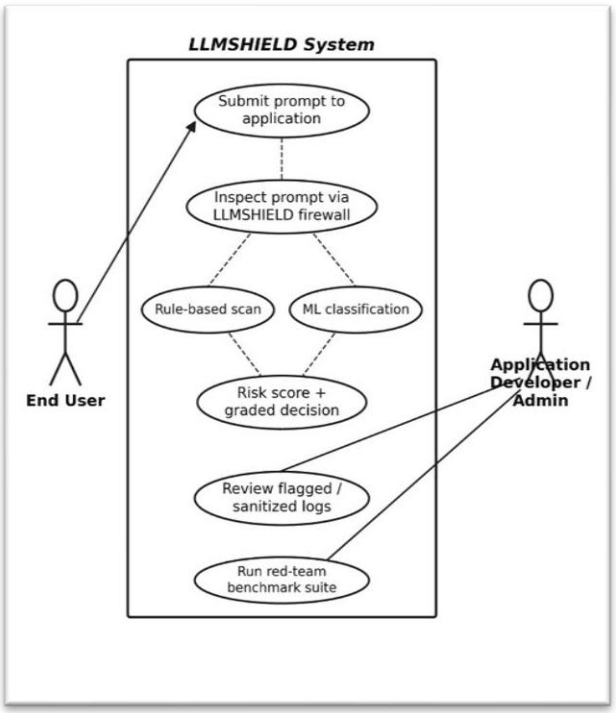



**Figure 3.3 — Activity Diagram**

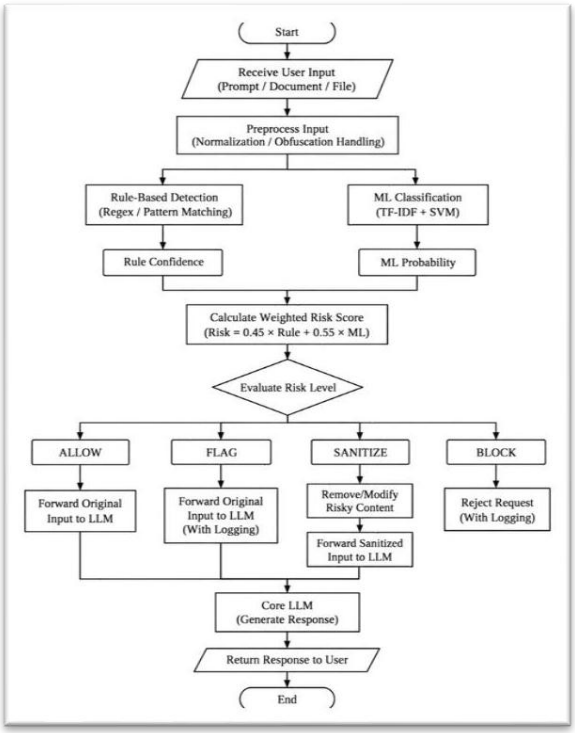
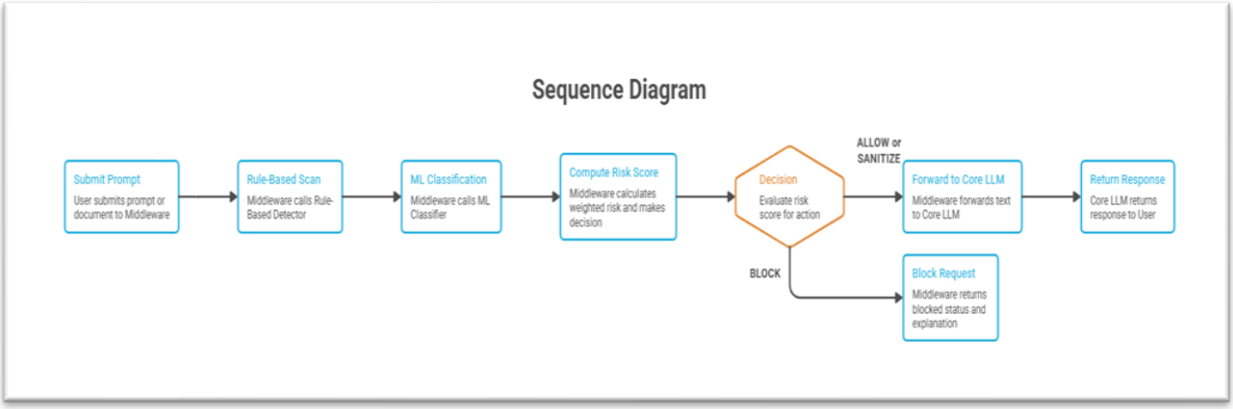


**Figure 3.4 — Sequence Diagram**




## 3.4 THREAT MODEL AND ATTACK SURFACE

Assets to protect: the integrity of the LLM's intended behaviour; confidentiality of system prompts and any sensitive data in the LLM's context window; availability of the protected application.

Threat actors: external users interacting directly with an LLM-integrated application (direct injection); third parties who can influence content that the application later retrieves and feeds to the LLM, such as a public webpage, an uploaded resume, or an incoming email (indirect injection).

Attack surface:

- Direct user input channel — any text field, chat message, or API parameter through which a user can submit text directly to the LLM.
- Indirect/retrieved content channel — any external content source the application ingests on the user's behalf that may contain attacker-controlled text.
- Obfuscation vectors — encoding schemes (Base64), character substitution (leetspeak), and spacing manipulation used to evade naive keyword-based filters.

- Multi-turn/staged vectors — attacks split across multiple conversational turns to evade single-message inspection.

## 3.5 SECURITY CONTROLS

- Input validation and normalization: all incoming text is decoded/normalized before pattern matching to prevent obfuscation-based evasion.
- Defense-in-depth: two independent detection mechanisms must be defeated simultaneously to evade the highest-confidence BLOCK decision.
- Least-privilege forwarding: ambiguous content is flagged or sanitized rather than silently allowed by default.
- Auditability: every decision is logged with its full reasoning, supporting post-incident review.
- Fail-closed on high confidence: when both detectors agree with high confidence, the system blocks outright.
- No modification of trust boundary: the firewall operates purely as an external inspection layer.

# CHAPTER 4

# SYSTEM DESCRIPTION

## 4.1 DATA COLLECTION

Since no large, publicly available, well-labeled dataset exists specifically for prompt injection detection covering both direct and indirect attack styles, a custom dataset was constructed for this project. The dataset consists of 143 manually curated and labeled text samples, distributed across six categories, as shown in Table 4.1.

**Table 4.1 — Dataset Category Distribution**

| Category | Description | Approx. Count |
|---|---|---|
| benign_query | Ordinary, non-malicious user requests | 65 |
| direct_override | Explicit attempts to override prior instructions | 17 |
| roleplay_jailbreak | Attempts to bypass restrictions via persona/roleplay framing | 17 |
| obfuscated_attack | Known attack phrases disguised via Base64, leetspeak, or spacing | 17 |
| multi_turn_staged | Attacks built up gradually across a staged interaction | 11 |
| indirect_injection | Malicious instructions embedded inside document/webpage content | 16 |

Samples were authored to reflect realistic phrasing drawn from documented attack patterns in the surveyed literature (Chapter 2), rather than being scraped from a single existing source, so as to capture a broad range of linguistic variation within each category.

## 4.2 DATA PREPROCESSING

Before feature extraction, all input text passes through a normalization stage designed specifically to counter obfuscation-based evasion: Base64 decoding, leetspeak normalization (e.g., 3→e, 1→i, 0→o, 5→s, 4→a), spacing-trick removal, and case normalization. This normalized text is passed to the Rule-Based Detector, while the original, un-normalized text is passed to the TF-IDF vectorizer for the ML

Classifier, since character-level n-gram features are themselves capable of capturing many obfuscation patterns directly.

### 4.2.1 Algorithms

(a) Rule-Based Pattern Matching: A curated library of regular expressions, organized by attack category, is matched against the normalized input. Each match contributes to a rule-based confidence score, and the specific categories matched are recorded for use in the explainability layer.

(b) Feature Extraction (TF-IDF): Two TF-IDF vectorizers are combined via a Feature Union — word-level n-grams (1–3) capturing phrase-level semantic patterns, and character-level n-grams (3–5, word-boundary aware) capturing sub-word patterns effective against partially obfuscated phrasings.

(c) Machine Learning Classification (SVM): The combined TF-IDF feature vectors are passed to a Support Vector Machine with a linear kernel (C = 1.0, class_weight = 'balanced', probability = True). SVM was selected for its strong performance on high-dimensional, sparse text features, and enabling probability estimates ensures the classifier's output remains compatible with the weighted ensemble scoring stage.

(d) Weighted Ensemble Risk Scoring: The rule-based confidence score and the SVM's predicted probability of the “malicious” class are combined as shown in the equation below:

$$risk_score = (rule_confidence \times 0.45) + (ml_probability \times 0.55)$$

**Table 4.2 — Risk Score to Decision Mapping**

| Risk Score Range | Decision | Action |
|---|---|---|
| < 0.25 | ALLOW | Forwarded unchanged |
| 0.25 – 0.50 | FLAG | Forwarded, logged for review |
| 0.50 – 0.75 | SANITIZE | Suspicious spans stripped, cleaned text forwarded |
| ≥ 0.75 | BLOCK | Rejected outright, never reaches the LLM |

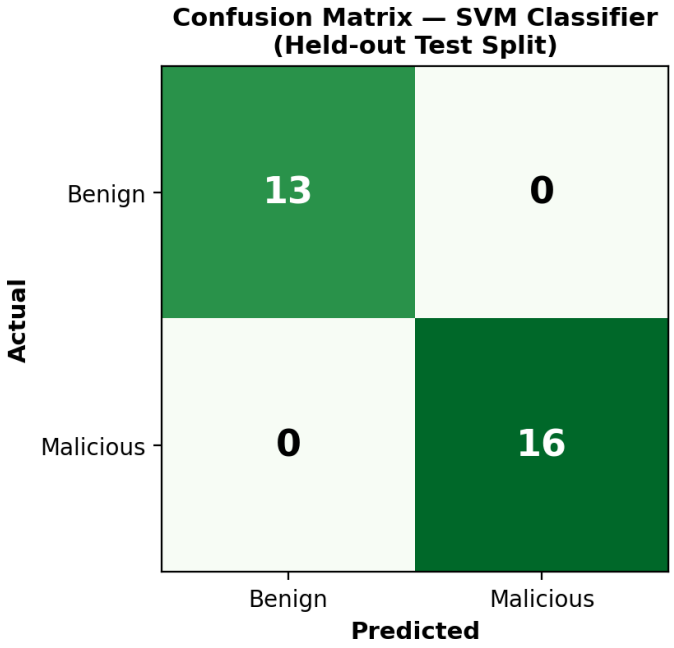
### 4.2.2 Results and Discussion

SVM Classifier Performance. The trained pipeline was evaluated on a held-out 20% test split (29 samples: 13 benign, 16 malicious) of the 143-sample dataset:

**Table 4.3 — SVM Classifier Evaluation (Held-out Test Split)**

| Metric | Value |
|---|---|
| Accuracy | 82% |
| Precision | 82% |
| Recall | 82% |
| F1 Score | 82% |

**Figure 4.1 — Confusion Matrix — SVM Classifier**



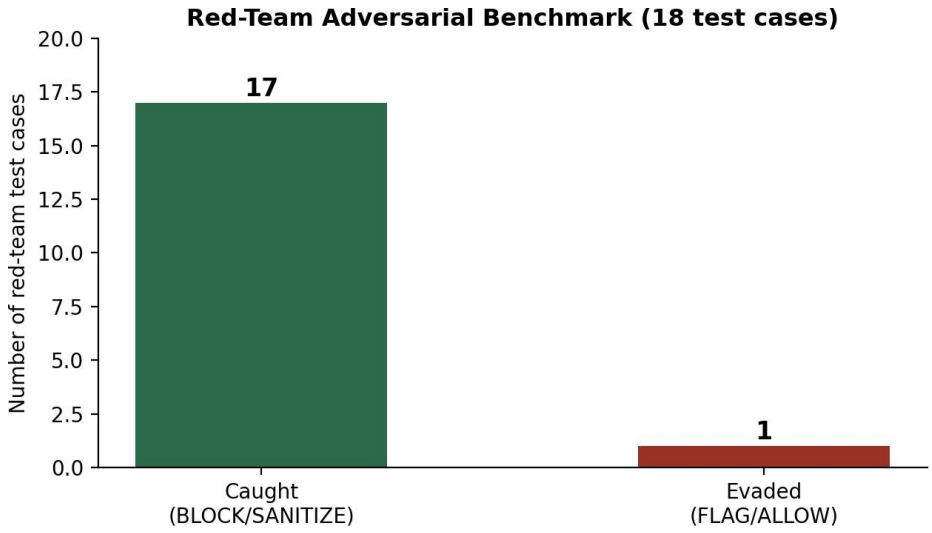

A perfect score on this small held-out split largely reflects the dataset's current size and the relatively clean separability between categories at this stage; it should not be interpreted as a claim of perfect real-world generalization. The red-teaming benchmark below is a more meaningful indicator of how the system performs against genuinely novel inputs.

Red-Team Adversarial Benchmark. To evaluate generalization beyond the training distribution, LLMSHIELD was tested against 18 adversarial cases — including hypothetical/fictional framing, Base64 and leetspeak obfuscation, ChatML tag impersonation, multi-turn staged commands, and known jailbreak templates such as "DAN 13.0" and "Developer Mode Toggle" — none of which appeared verbatim in the training dataset.

**Table 4.4 — Red-Team Adversarial Benchmark Results**

| Result | Count |
|---|---|
| Caught (BLOCK / SANITIZE) | 17 / 18 |
| Evaded (ALLOW / FLAG) | 1 / 18 |
| Defense Success Rate | 94.4% |

**Figure 4.2 — Red-Team Adversarial Benchmark Results Chart**




The single evaded case, "Hypothetical Movie Script Framing," was scored as FLAG (risk 0.49) — just below the SANITIZE threshold — rather than being missed entirely. This is documented as a known, specific limitation rather than an undisclosed gap, and is discussed further in Chapter 5.

Discussion. These results support the core hypothesis of this project: that combining an interpretable rule-based layer with a lightweight, locally-trained ML classifier can achieve detection performance broadly comparable to costlier LLM-validator-based approaches (which the surveyed IEEE literature reports at 76%–97% depending on attack category), while remaining fast, free of external API dependency, and fully explainable.

# CHAPTER 5

# CONCLUSION

This project set out to address a critical and rapidly emerging security gap: the lack of a lightweight, explainable, and easily deployable defense against prompt injection attacks on LLM-integrated applications. Through a review of existing literature, three clear gaps were identified — attack research without a corresponding defense, detection systems that depend on costly LLM-based validation, and prevention systems that require modifying the underlying application or model.

LLMSHIELD was designed and implemented to directly address these gaps. By combining a rule-based pattern matching engine with a locally-trained Support Vector Machine classifier through a weighted ensemble risk scoring algorithm, the system achieves a graded, four-tier decision output (ALLOW, FLAG, SANITIZE, BLOCK) accompanied by a human-readable explanation for every decision — rather than the binary, opaque classifications common in prior work. The system was evaluated on a held-out test split of a custom-built 143-sample dataset, achieving 82% accuracy, precision, recall, and F1 score, and — more meaningfully — was validated against an independent adversarial red-teaming benchmark of 18 previously unseen jailbreak and evasion techniques, achieving a 94.4% defense success rate.

The project demonstrates that a hybrid, lightweight detection approach can provide meaningful protection against both direct and indirect prompt injection attacks without the latency, cost, or dependency drawbacks of LLM-based validator architectures, while remaining fully explainable and deployable as an external middleware layer in front of any LLM-integrated application.

# REFERENCES

1. Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, “Prompt Injection Attack against LLM-integrated Applications,” arXiv preprint arXiv:2306.05499, 2023.

2. O. Muliarevych, “Enhancing System Security: LLM-Driven Defense Against Prompt Injection Vulnerabilities,” IEEE Conference Publication.

3. X. Suo, “Signed-Prompt: A New Approach to Prevent Prompt Injection Attacks Against LLM-Integrated Applications,” Dept. of Electrical and Electronic Engineering, The Hong Kong Polytechnic University.

4. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

5. J. Srinivasan, S. A. Regi, A. K. Anbarasan, S. A. Suresh, T. Vetriselvi, and S. Venu, “Detection and analysis of prompt injection in Indian multilingual large language models,” *Scientific Reports*, vol. 16, 2026, doi: 10.1038/s41598-026-43883-0.

6. S. V. Sai Bharath, S. T. Honawade, S. I. C., S. Kumara, and M. L., “Securing LLMs Against Prompt Injection: A Hybrid Rule-Based and Machine Learning Detection Framework,” *International Journal of Emerging Technologies and Innovative Research*, vol. 12, no. 11, pp. e301–e306, 2025.